Priority Optimization for Sequential Multi-Agent Path Planning

A THESIS
SUBMITTED TO THE FACULTY
OF THE UNIVERSITY OF MINNESOTA
BY

Tianning Li

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE DEGREE OF
MASTER OF SCIENCE

Advisor: Yue Yu

December, 2025

© Tianning Li 2025
ALL RIGHTS RESERVED

Acknowledgements

I would like to express my sincere gratitude to my advisor, Professor Yue Yu, for his continuous guidance, patience, and encouragement throughout this project. His insights have shaped both the technical direction of this thesis and my development as a researcher.

I am also grateful to my committee members, Professor Timothy Kowalewski and Professor Aryan Deshwal, for their valuable feedback and support.

I would like to thank Travis Henderson, my program advisor, for his continual guidance and administrative support throughout my studies.

Finally, I acknowledge the use of AI-assisted tools, including ChatGPT, which supported language refinement and clarity under my full supervision.

Dedication

To my parents and loved ones, whose love and belief in me made this journey possible.

Abstract

Multi-Agent Path Planning (MAPP) is a fundamental problem in robotics and logistics, requiring the generation of collision-free trajectories for multiple agents operating in a shared environment. In prioritized MAPP, agents plan sequentially according to a prescribed priority order, and the path of each higher-priority agent becomes a time-dependent constraint for all remaining agents. Under dynamic goal blocking—where each finished agent's goal becomes a temporary obstacle—the quality and feasibility of the final joint solution depend strongly on the chosen execution order.

This thesis formulates priority-order selection as a black-box optimization problem over permutations in prioritized MAPP. Each ordering is evaluated using a composite metric that integrates path-efficiency cost and safety-related congestion cost into a single execution-quality measure. Four methods—Random Sampling, Simulated Annealing (SA), Genetic Algorithm (GA), and Bayesian Optimization (BO)—are applied to search the priority-permutation space across grid environments of varying difficulty and agent counts (10–40 agents). Random sampling reveals the natural distribution of feasible and infeasible solutions, as well as the variability among valid priority orders. The optimization algorithms aim to more systematically locate high-quality regions of the permutation space.

The results show that all optimization methods consistently avoid catastrophic priority assignments that arise in random sampling. SA provides consistent early-stage improvement but displays variability across trials due to its single-trajectory search. GA offers more stable performance through population diversity, although it may plateau depending on the environment. BO demonstrates robust model-guided exploration and frequently identifies competitive solutions, albeit with higher computational cost. Overall, the study highlights both the promise and the practical limitations of optimization-based priority selection in prioritized MAPP. The findings motivate future work on richer neighborhood operators for priority-permutation search, advanced variants of SA and GA tailored to priority planning, and joint optimization with dynamic replanning.

Contents

Acknowledgements **i**

Dedication **ii**

Abstract **iii**

List of Tables **vi**

List of Figures **vii**

1 Introduction **1**

1.1 Research Contributions 2

1.2 Organization 2

2 Literature Review **3**

2.1 Multi-Agent Priority Path Planning 3

2.2 Optimization Methods in Multi-Agent Path Planning 5

2.3 Limitations of Existing Results 7

3 Priority Optimization in Multi-Agent Path Planning **8**

3.1 Problem Formulation 8

3.1.1 Agents and Obstacles 8

3.1.2 Priority Order Definition 9

3.1.3 Metrics 9

3.1.4 Composite Optimization Function 10

3.2 Optimization Algorithms 11

3.2.1 Sequential Time-Expanded Planning 11

3.2.2 Local Operators 12
3.2.3 Random Sampling (Baseline) 12
3.2.4 Simulated Annealing (SA) 13
3.2.5 Genetic Algorithm (GA) 13
3.2.6 Bayesian Optimization (BO) 13
3.3 Post-Hoc Evaluation Metrics 14

4 Results **15**
4.1 Experimental Setup 15
4.2 Evaluation and Visualization Protocol 16
4.3 Medium-Scale Environments 19
4.3.1 10-Agent Environment 19
4.3.2 20-Agent Environment 22
4.4 Large-Scale Environments 25
4.4.1 30-Agent Environment 25
4.4.2 40-Agent Environment 28

5 Discussion and Conclusion **31**
5.1 Interpretation of Results 31
5.1.1 Simulated Annealing 31
5.1.2 Genetic Algorithm 32
5.1.3 Bayesian Optimization 32
5.2 Limitations 32
5.3 Future Work 33
5.4 Conclusion 33

References **35**

Appendix A. Glossary and Acronyms **38**
A.1 Glossary 38
A.2 Acronyms 39

List of Tables

4.1 Performance comparison cross obstacle types for the 10-agent scenario. . . . 22
4.2 Performance comparison cross obstacle types for the 20-agent scenario. . . . 24
4.3 Performance comparison cross obstacle types for the 30-agent scenario. . . . 27
4.4 Performance comparison cross obstacle types for the 40-agent scenario. . . . 30
A.1 Acronyms 39

List of Figures

4.1 Environment layouts for the 10-agent scenario (obstacle types A, B, C). . . 19
4.2 Random sampling results for the 10-agent scenario: (a–c) valid–invalid histograms; (d–f) valid-only objective distributions. 20
4.3 Optimization results for the 10-agent scenario under obstacle type C: (a–c) feasibility structure; (d–f) mean best-so-far convergence curves with one-standard-deviation bands. 21
4.4 Environment layouts for the 20-agent scenario (obstacle types A, B, C). . . 22
4.5 Random sampling results for the 20-agent scenario: (a–c) valid–invalid histograms; (d–f) valid-only objective distributions. 23
4.6 Optimization results for the 20-agent scenario under obstacle type B: (a–c) feasibility structure; (d–f) mean best-so-far convergence curves with one-standard-deviation bands. 24
4.7 Environment layouts for the 30-agent scenario (obstacle types A, B, C). . . 25
4.8 Random sampling results for the 30-agent scenario: (a–c) valid–invalid histograms; (d–f) valid-only objective distributions. 26
4.9 Optimization results for the 30-agent scenario under obstacle type B: (a–c) feasibility structure; (d–f) mean best-so-far convergence curves with one-standard-deviation bands. 27
4.10 Environment layouts for the 40-agent scenario (obstacle types A, B, C). . . 28
4.11 Random sampling results for the 40-agent scenario: (a–c) valid–invalid histograms; (d–f) valid-only objective distributions. 28
4.12 Optimization results for the 40-agent scenario under obstacle type A: (a–c) feasibility structure; (d–f) mean best-so-far convergence curves with one-standard-deviation bands. 29

Chapter 1

Introduction

Multi-Agent Path Planning (MAPP) addresses the problem of planning collision-free trajectories for multiple agents moving on a shared grid. Classical MAPP approaches are generally categorized into two families: decoupled methods such as prioritized planning (e.g., Cooperative A*, CA*), and coupled methods such as Conflict-Based Search (CBS). In prioritized planning, agents plan sequentially according to a chosen priority order. Higher-priority agents commit to their paths first, and their trajectories become time-dependent obstacles for lower-priority agents. As a result, the quality and feasibility of the final joint solution depend strongly on the selected priority order.

Since the priority space grows factorially with the number of agents and each evaluation requires computing time-extended, collision-free trajectories, exhaustive search becomes computationally infeasible. Priority selection therefore forms a discrete combinatorial optimization problem over a large permutation space.

This thesis treats priority assignment as a black-box optimization task. Each priority permutation is evaluated using a composite objective measuring both overall efficiency (path length cost) and safety-related congestion (waiting and spatial interference). The goal is not to identify provably optimal orders, but to examine whether general-purpose optimization methods can consistently improve solution quality over random selection when applied to this discrete search problem.

To this end, three optimization algorithms—Simulated Annealing (SA), Genetic Algorithm (GA), and Bayesian Optimization (BO)—are applied to the priority-order search problem. SA and GA are classical metaheuristics widely used in discrete combinatorial domains. BO is traditionally designed for continuous spaces, but recent work on permutation

kernels and local-neighborhood sampling motivates an investigation of its behavior in this setting. A random sampling baseline is included to illustrate the natural variability of the objective landscape and the existence of both extremely poor and high-quality priority assignments.

1.1 Research Contributions

This thesis introduces and evaluates an optimization-based framework for priority-order selection in prioritized MAPP. The main contributions are:

- Formulates priority-order selection in prioritized MAPP as a discrete, black-box optimization problem using a composite efficiency–safety metric computed under dynamic goal blocking.
- Implements and evaluates three global optimization methods (SA, GA, BO) using shared local-neighborhood operators for permutation search.
- Provides a systematic comparison against random baselines across multiple environments, analyzing convergence behavior, solution quality, and variability.
- Offers insights into how global optimization behaves in permutation-based priority planning under dynamic goal blocking, and identifies limitations and opportunities for future improvements.

1.2 Organization

The thesis is organized as follows:

- Chapter 2 reviews related work in prioritized MAPP and optimization strategies in MAPP.
- Chapter 3 presents the modeling assumptions, the definition of priority permutations, the time-expanded prioritized planner, the composite evaluation metric, and the implementations of the optimization algorithms.
- Chapter 4 reports the experimental environments and results, including random-sampling distributions, convergence behavior, and performance statistics.
- Chapter 5 discusses interpretation of results, methodological limitations, possible improvements, and concluding remarks.

Chapter 2

Literature Review

2.1 Multi-Agent Priority Path Planning

Multi-agent path planning (MAPP) addresses the problem of finding collision-free paths for multiple agents moving on a shared discrete graph. Each agent has a unique start and goal location, and moves in discrete time steps by staying in place or moving to an adjacent vertex. Collisions occur when two agents occupy the same vertex at the same time (vertex collision) or traverse the same edge in opposite directions during the same timestep (edge collision). MAPP is NP-hard to solve optimally and appears in many practical domains such as warehouse automation and multi-robot coordination. Following the standard MAPP formulation used in classical solvers such as CBS and PBS, an agent that reaches its goal remains there permanently. Vertex-collision and edge-collision constraints apply until the agent arrives at the goal, after which its location is fixed. Consistent with common MAPP benchmarks, the cost of a path excludes any timesteps during which the agent waits at the goal [1].

Prioritized planning is one of the simplest and most widely used decoupled approaches for solving MAPP. In this framework, each agent is assigned a unique priority and plans its path sequentially from highest to lowest priority. Each agent computes a shortest path that avoids both static obstacles and the already-planned paths of all higher-priority agents. The idea of using agent priorities was originally introduced by Erdmann and Lozano-Pérez [2] and was later widely adopted in MAPP through algorithms such as Cooperative A* [3]. Modern MAPP applications continue to rely on prioritized planning due to its efficiency and robustness in practice [4, 5].

Despite its efficiency, priority-based MAPP is highly sensitive to the chosen priority ordering. Different orderings can lead to dramatically different solution qualities, ranging from efficient low-cost solutions to deadlocks or complete failure. Prior work shows that fixed priority orders are often incomplete and may fail even on solvable instances [4]. Ma et al. demonstrate that small changes in the priority order can significantly impact both the success rate and cost of prioritized planning [6]. Furthermore, recent learning-based evaluations show that running prioritized planning over 100 random priority orderings can produce a wide range of solution costs and failure cases for the same MAPP instance [5]. These studies highlight that priority selection is a critical factor in the overall performance of priority-based MAPP methods.

Several advanced methods have been developed to address the limitations of fixed priority assignments. Early work by Bennewitz, Burgard, and Thrun proposed optimizing priority schemes using randomized hill climbing with random swaps, showing that even simple local search can improve success rates and reduce path costs in prioritized planning [7,8]. However, these methods remain local and do not systematically explore the global permutation space. Priority-Based Search (PBS) [6] was later introduced to systematically explore the space of partial priority orderings by two levels–a high-level search constructs a priority tree to resolve conflicts by imposing ordering constraints between agents while a low-level planner computes individual paths that satisfy the imposed priorities, avoiding the drawbacks of committing to a single full order. More recently, Greedy Priority-Based Search (GPBS) [9] extended PBS with greedy node selection, partial expansions, and restart strategies to achieve higher performance under strict time budgets. Other variants, such as improved prioritized trajectory-coordination methods [4] and learning-based priority selection [5], further highlight ongoing efforts to mitigate the sensitivity of prioritized planning to the chosen ordering.

The performance of prioritized MAPP depends heavily on the chosen priority ordering, and existing methods typically select priorities heuristically, randomly, or manually. Because different orderings can lead to widely varying solution qualities, the priority-assignment process can be naturally viewed as an optimization problem over the space of agent permutations. This perspective motivates the use of numerical optimization methods to find effective priority orders. Prior work in combinatorial optimization shows that simulated annealing [10] and genetic algorithms [11] are effective tools for exploring large permutation spaces, and Bayesian optimization has recently been extended to permutation

domains through specialized kernels and acquisition strategies [12]. The remainder of this thesis examines these three optimization methods to explore the priority space and improve solution quality in prioritized MAPP.

2.2 Optimization Methods in Multi-Agent Path Planning

Simulated Annealing (SA) is a stochastic optimization method inspired by physical annealing. It perturbs candidate solutions through local modifications and accepts changes according to a temperature-controlled probability rule, enabling escape from local minima in complex search landscapes [13]. SA has been applied to multi-robot path planning as a global-search method capable of improving path quality and resolving difficult coordination constraints. W. Shi et al. employ SA to multi-robot systems with Boolean task specifications, where SA iteratively refines robot trajectories to satisfy logical constraints while minimizing total travel distance; their results show that SA produces lower computational cost and higher solution quality compared to conventional methods [14]. K. Shi et al. integrate SA into a priority-based dynamic coordination framework for multi-robot path planning, demonstrating that their work provides better overall performance than other dynamic path planning approaches while remaining reliable across both dynamic and static environments [15]. Alarabi et al. combine SA with potential-field navigation to allow multiple robots to escape local minima and maintain safe inter-robot distances, showing improved robustness in cluttered environments [16]. Collectively, these works demonstrate that SA enhances multi-robot path planning by providing a flexible mechanism for exploring non-convex search spaces, refining suboptimal trajectories, and improving safety and coordination among multiple agents.

Genetic Algorithms (GA) are population-based metaheuristics that search complex spaces through iterative variation and selection. It has been widely applied to multi-robot path planning because of their ability to explore high-dimensional, non-convex search spaces and generate collision-free paths under complex constraints. Qu et al. propose a co-evolutionary improved GA in which multiple subpopulations evolve in parallel and exchange information asynchronously; their cooperation mechanism reduces path conflicts and improves convergence compared to standard GA approaches [17]. Nazarahari et al. develop an enhanced GA that integrates multiple customized crossover and mutation operators, multi-objective path quality metrics (length, smoothness, safety), and a collision-removal operator, showing that GA can effectively optimize path quality for multiple robots in

continuous environments [18]. Kala introduces a co-evolutionary genetic programming scheme where each robot evolves its own path while interacting implicitly with others, enabling adaptation to multi-robot coordination challenges and avoidance of collisions in cluttered environments [19]. Together, these studies indicate that GA provides a flexible evolutionary framework for multi-robot coordination, capable of improving solution quality, enhancing robustness in dense environments, and outperforming several classical and evolutionary baselines.

Bayesian Optimization (BO) is a model-based global optimization method designed for expensive black-box functions, and recent advances have extended BO from continuous spaces to discrete and permutation structures. In combinatorial optimization, permutation-aware kernels and acquisition functions allow BO to model uncertainty over non-Euclidean spaces such as agent orderings [12]. BO has also been applied to the generation of multi-agent trajectory, where Ryou et al. use modular BO to optimize cooperative multi-agent motion in continuous configuration spaces [20]. These developments suggest that BO can serve as a sample-efficient tool for exploring complex decision spaces in multi-agent systems, including the priority-permutation space in prioritized MAPP.

Multi-criteria evaluation has become increasingly important in multi-robot and multi-agent path planning, as real-world systems must often be assessed along multiple dimensions beyond classical measures such as path length or makespan. For example, Nazarahari et al. consider metrics such as path length, smoothness, and safety when optimizing continuous multi-robot trajectories using an enhanced genetic algorithm [18]. Wang et al. jointly evaluate distance, congestion, travel time, energy consumption, and safety in multi-vehicle urban mobility scenarios [21]. Ho and Nakadai propose a preference-based extension of MAPP that evaluates solutions according to efficiency, safety, and smoothness [22].

These works demonstrate the broad relevance of using multiple metrics to assess solution quality in multi-agent systems. Motivated by these formulations, this thesis adopts a unified scalar objective that integrates several evaluation metrics—specifically efficiency and coordination difficulty—within a single composite formulation, as detailed in the Methods section.

2.3 Limitations of Existing Results

Although priority assignment is one of the most influential factors in prioritized MAPP, existing approaches do not treat priority selection as a global optimization problem. Instead, they rely on heuristics, random orders, supervised learning, or partial-order search; none explicitly explore the factorial space of priority permutations using general-purpose optimization algorithms.

Furthermore, while several studies evaluate MAPP solutions using multiple criteria such as efficiency or safety, these evaluations are typically performed after planning and are not used to guide the search for improved priority orders. Consequently, how systematic priority optimization influences execution quality has not been examined in detail.

No prior work applies global optimization strategies—such as SA, GA, or BO—to directly search over priority permutations in prioritized MAPP. This thesis addresses this gap by evaluating whether such optimization methods can improve execution quality and reduce coordination difficulty under sequential and goal-blocking constraints.

Chapter 3

Priority Optimization in Multi-Agent Path Planning

3.1 Problem Formulation

The environment is modeled as a discrete two-dimensional grid world represented by a graph

$$G = (V, E), \qquad V = \{(i, j) \mid i \in \{0, \ldots, M-1\},\ j \in \{0, \ldots, N-1\}\}.$$

with four-connected edges between neighboring cells. The grid size $M \times N$ defines the spatial boundary of the environment.

3.1.1 Agents and Obstacles

Each agent r_i is assigned an initial position s_i and a goal position g_i. Two types of obstacles restrict the movement of agents:

- **Static obstacles** $O_s \subset V$: fixed and time-invariant blocked cells.
- **Dynamic obstacles.** Two types arise during sequential planning:
  - *Reservation table obstacles* $O_r(t) \subset V$: occupied cells indexed in time generated by higher-priority agents' trajectories. These cells are blocked only at specific timestep when an agent occupies them.
  - *Permanent goal-blocking obstacles* $O_g(t) \subset V$: once an agent reaches its goal, that goal cell becomes permanently blocked for all remaining agents. Once an agent reaches its goal, that goal cell becomes permanently blocked for all remaining agents and remains unavailable for all $t' \geq t$.

The feasible region at time t is therefore given by

$$V_t^{\text{free}} = V \setminus \big(O_s \ \cup \ O_r(t) \ \cup \ O_g(t)\big),$$

which excludes both static barriers, time-indexed reservation-table conflicts, and permanently blocked goal cells introduced by higher-priority agents.

3.1.2 Priority Order Definition

A priority order is defined as a permutation

$$\pi = (\pi_1, \pi_2, \ldots, \pi_N),$$

where π_i denotes the index of the agent assigned to the i-th planning position. That is, the agent with priority rank π_1 plans first, followed by the agent π_2, and so on. Under this ordering, earlier agents plan their trajectories independently, while later agents must treat the trajectories of all higher-priority agents as dynamic obstacles in space–time.

Example. Consider three agents $\{r_1, r_2, r_3\}$ and a priority order

$$\pi = (2, 3, 1).$$

This means:

- $\pi_1 = 2$: agent r_2 plans first;
- $\pi_2 = 3$: agent r_3 plans second, avoiding r_2's trajectory;
- $\pi_3 = 1$: agent r_1 plans last, avoiding the trajectories of both r_2 and r_3.

Thus, the permutation π describes the *planning order*, not the physical positions of the agents. Different permutations produce different space–time constraints and therefore different multi-agent workloads and total objective values.

3.1.3 Metrics

For a fixed priority order π, each agent r_i obtains a discrete-time path represented as

$$\mathcal{P}_i = \{(x_i(t), y_i(t), t)\}_{t=0}^{L_i},$$

where L_i denotes the timestep at which the agent reaches its goal. Because each timestep incurs uniform effort, the individual path cost is simply

$$C_i = L_i.$$

Efficiency Metrics

The overall travel effort under ordering π is measured by the total cost

$$C(\pi) = \sum_{i=1}^{N} C_i,$$

where larger values indicate greater cumulative travel effort across all agents.

Safety Metrics

Efficiency alone does not reflect the difficulty of executing the coordinated multi-agent paths. To capture interaction-induced delays and proximity-related risks, two additional safety metrics are defined.

Waiting time. This metric counts the total number of timesteps during which an agent remains stationary, typically due to congestion or conflict-avoidance behavior.

$$W(\pi) = \sum_{i=1}^{N} \left|\{t \mid (x_i(t), y_i(t), t) = (x_i(t-1), y_i(t-1), t-1)\}\right|.$$

Congestion penalty. This metric counts timesteps where two agents occupy adjacent cells (Manhattan distance ≤ 1), indicating potential spatial congestion.

$$G(\pi) = \sum_{i<j} \left|\left\{t \;\middle|\; \left\|(x_i(t), y_i(t), t) - (x_j(t), y_j(t), t)\right\|_1 \leq 1\right\}\right|.$$

Combined safety metric. The overall safety metric is defined as:

$$S(\pi) = W(\pi) + G(\pi),$$

which represents the cumulative coordination difficulty generated by the priority order π.

3.1.4 Composite Optimization Function

This work evaluates each priority order using a scalar Composite Objective Function (COF) that aggregates the main performance criteria of sequential planning.

The COF is defined as

$$J(\pi) = \alpha\, C(\pi) + (1-\alpha)\, S(\pi),$$

where $C(\pi)$ measures overall workload and $S(\pi)$ captures interaction-related interference.

A fixed value of $\alpha = 0.5$ is used for all experiments. This equal weighting reflects two design considerations:

1. **Balanced interpretation:** Both workload and interference contribute significantly to execution quality, and neither should dominate the evaluation.
2. **Consistency across scenarios:** Using a fixed global parameter enables fair comparisons of SA, GA, BO, and random baselines across all environments.

The COF produces a single scalar value that is suitable for optimization in permutation spaces and is consistent across all algorithms evaluated in this study.

3.2 Optimization Algorithms

Since the permutation space Π_N grows factorially with the number of agents, evaluating all priority orders is computationally infeasible. This study applies Simulated Annealing (SA), the Genetic Algorithm (GA), and Bayesian Optimization (BO) to identify a priority order π that yields the smallest objective value $J(\pi)$ as defined in Equation (3.1.4). Each optimization algorithm uses a stopping rule that balances computational cost with solution quality. A random sampling baseline is also included to provide a reference distribution for comparison.

3.2.1 Sequential Time-Expanded Planning

In this work, the term *planner* refers to the single-agent path planner that computes a collision-free trajectory for one agent at a time, following a fixed priority order π. The planner uses a *time-expanded A^* search* meaning that the search graph explicitly represents both spatial positions and timesteps. In this formulation, each node corresponds to a state (x, y, t) and edges represent feasible movements between timesteps. All higher-priority agents' trajectories are incorporated as time-indexed constraints through a reservation table.

For each agent, the planner constructs a time-expanded graph whose states are defined as

$$(x, y, t),$$

where (x, y) is a grid cell and t is a discrete timestep. At each timestep, an agent may

remain stationary or move to a 4-neighbor cell:

$$(x, y, t) \to (x', y', t+1), \qquad (x', y') \in \mathcal{N}(x, y) \cup \{(x, y)\}.$$

Feasibility constraints. A transition into $(x', y', t+1)$ is allowed only if the destination is free of all obstacle types:

$$\begin{aligned} (x', y') &\notin O_s, \\ (x', y', t+1) &\notin O_r(t+1), \\ (x', y', t+1) &\notin O_g(t+1), \\ (x', y', t+1) &\notin \text{Reserved}. \end{aligned}$$

The set *Reserved* stores space–time reservations generated by previously planned agents and prevents both vertex conflicts and edge-swap conflicts.

Sequential dependence. Because each agent must avoid the committed trajectories of all higher-priority agents, the feasible region for later agents becomes increasingly restrictive. This dependence on the priority order π induces a complex and highly non-convex optimization landscape over permutations.

3.2.2 Local Operators

All optimization algorithms generate new candidate priority orders using one of the following local operators. At each iteration, an operator is selected at random, and its internal parameters (such as the positions i and j) are also sampled randomly.

- **Adjacent swap:** exchange agents at positions i and $i+1$.
- **Random swap:** exchange agents at two randomly chosen positions.
- **3-cycle rotation:** cyclically rotate agents at positions $(i, i+1, i+2)$.
- **Insert move:** remove the agent at position i and insert it at position j.

These operators define the discrete neighborhood structure used in SA, GA, and BO. The random selection of operators ensures that the search explores different regions of the permutation space.

3.2.3 Random Sampling (Baseline)

Random sampling evaluates a fixed number of uniformly sampled permutations(e.g., 40) from Π_N. Each sampled priority order is evaluated using the metrics in Section 3.1.3 and

the COF in Section 3.1.4. This baseline characterizes the difficulty of the landscape and provides reference performance for the optimized methods.

3.2.4 Simulated Annealing (SA)

The SA performs a stochastic local search using randomly selected local operators defined in Section 3.2.2. Given a current priority order π, a neighbor π' is accepted deterministically if $J(\pi') < J(\pi)$, and with probability

$$p = \exp\left(\frac{J(\pi) - J(\pi')}{T}\right)$$

otherwise. The temperature follows a geometric decay:

$$T' = \alpha_T T, \qquad 0 < \alpha_T < 1.$$

The temperature parameter T controls the probability of accepting worse solutions, allowing exploration of the permutation space. The SA algorithm terminates after a fixed number of iterations.

3.2.5 Genetic Algorithm (GA)

The GA evolves a population of priority orders using fitness evaluation, selection, crossover, and mutation. A fixed population size is maintained throughout evolution. Each generation consists of:

1. **Fitness evaluation:** Each π is scored using $J(\pi)$.
2. **Selection:** The top 50% are retained.
3. **Order-preserving crossover:** A child permutation inherits a prefix from one parent and fills the rest according to the ordering in the second parent.
4. **Mutation:** With small probability, a randomly selected local operator (Section 3.2.2) is applied to introduce diversity.
5. **Termination:** The GA algorithm terminates after a fixed number of generations.

3.2.6 Bayesian Optimization (BO)

The BO algorithm is adapted to permutation space by employing a Gaussian Process (GP) with a Kendall–tau kernel. The BO loop proceeds as follows:

1. **Initial sampling:** A small set of random permutations is evaluated using $J(\pi)$.
2. **Permutation kernel:**

$$k(\pi_i, \pi_j) = \exp(-\lambda\, d_\tau(\pi_i, \pi_j)),$$

   where d_τ is the Kendall–tau distance.
3. **Candidate generation:** A set of neighboring permutations is generated using the local operators in Section 3.2.2.
4. **GP modeling:** The surrogate model predicts the posterior distribution of $J(\pi)$.
5. **Acquisition via Thompson Sampling:** A candidate with the lowest sampled posterior value is selected for evaluation.
6. **Update:** The surrogate is updated with the new observation.

The BO algorithm terminates after a fixed number of iterations.

3.3 Post-Hoc Evaluation Metrics

After optimization, each method's best priority order is evaluated using the metrics defined in Section 3.1.3. These metrics summarize the efficiency, coordination difficulty, and computational cost of the resulting solution:

- total workload $C(\pi)$,
- combination of structural interference cost $S(\pi)$,
- composite objective value $J(\pi)$,
- total evaluation runtime.

Chapter 4

Results

4.1 Experimental Setup

All experiments are conducted in a discrete grid-world environment where agents move synchronously in unit time steps on a 4-connected grid (up, down, left, right). A collision occurs when two agents attempt to occupy the same cell or traverse the same edge in opposite directions at the same timestep. For each agent-scale scenario, the start and goal locations for all agents are fixed to ensure comparability across different obstacle configurations.

To study how map structure influences priority optimization, three obstacle configurations were created for each scale (10, 20, 30, and 40 agents). Only the obstacles patterns vary; all other elements—starts, goals, grid size, and movement rules—remain identical. The three obstacle types are:

- Type A (Sparse): mostly open space with minimal bottlenecks.
- Type B (Moderate): a mixture of open regions and narrow passages that induces mild congestion.
- Type C (Dense/Structured): long block walls, funnel-like corridors, and multiple choke points that significantly constrain movement.

Obstacle difficulty is qualitatively categorized as low, medium, and high based on obstacle density, number of narrow passages, and the expected level of agent congestion. This classification style follows standard practice in MAPP benchmark design.

4.2 Evaluation and Visualization Protocol

This section describes the evaluation procedure, metrics, interpretation of invalid cases, and the visualization framework used throughout all agent-scale experiments (10, 20, 30, and 40 agents). These explanations appear only once to avoid repetition and to ensure consistent interpretation of the subsequent result figures and tables. All experiments were executed on a 14-inch MacBook Pro (Apple M4 Max, 36 GB RAM), ensuring consistent computational conditions across all evaluations.

Evaluation Procedure

For each agent scale and obstacle configuration, four methods—Random Sampling, Simulated Annealing (SA), Genetic Algorithm (GA), and Bayesian Optimization (BO)— were evaluated under fixed parameter settings.

Each method was executed for 30 independent trials. In every trial, the algorithm was given the same number of iterations, and the best priority order identified within that trial was recorded.

The random baseline follows the same iteration limit as the optimized methods. Instead of averaging all random samples, each trial reports the best random priority order found within the allotted iterations, providing a fair reference for comparison with SA, GA, and BO.

Extracted Metrics

For each trial, the following quantities were extracted from the simulated execution:

- **Efficiency cost** $C(\pi)$: total path length summed over all agents.
- **Safety cost** $S(\pi)$: sum of waiting time and congestion penalties.
- **Composite objective** $J(\pi) = \alpha C(\pi) + (1 - \alpha)S(\pi)$ with $\alpha = 0.5$.
- **Runtime**: total wall-clock time required for the algorithm to evaluate a priority order, including all planning and update operations.

Invalid Solutions

Results marked as *invalid* indicate that the planner was unable to compute collision-free paths for all agents within the allowed planning process. Such cases correspond to priority

orders for which no feasible joint solution exists. Invalid outcomes appear only in the feasibility plots—primarily the random baseline and the iteration-wise validity plots for SA, GA, and BO.

Visualization Overview

To present the behavior of the four methods (Random, SA, GA, BO) in a consistent and interpretable manner, three categories of visualizations are used throughout the results section. These appear in the same structured format for every agent scale (10, 20, 30, and 40 agents). The visualizations explain feasibility characteristics, search dynamics, and performance outcomes, respectively.

Random Sampling Visualization. Random sampling serves as an uninformed baseline for understanding the structure of the priority–permutation space. Two plots are generated for each obstacle type:

1. **valid–invalid histogram (per trial).** This plot shows, for each of the 30 trials, how many randomly sampled priority orders were feasible versus infeasible. Random sampling is given the same number of iterations as the three optimization methods (SA, GA, and BO), ensuring a fair comparison of feasibility rates across methods. The histogram reveals how often the planner fails due to collisions under uninformed sampling, thus exposing the feasibility landscape of each environment.
2. **Scatter plot of valid samples only.** This plot displays the objective values of all feasible priority orders collected across the 30 trials, using the same iteration budget as in the optimization methods. By showing only valid evaluations, it illustrates the natural spread, variability, and difficulty of the search landscape without optimization guidance.

Together, these plots answer:

- How feasible is the environment under random sampling?
- How widely do objective values vary among feasible solutions?
- How challenging is the ordering space before optimization is applied?

Optimization Methods Visualization (SA, GA, BO). For each agent scale, only one representative obstacle type is used to illustrate the search behavior of SA, GA, and BO,

avoiding redundancy while still revealing typical algorithmic progression. Two plots per method are shown:

1. **Validity-by-iteration plot.** A stacked bar plot showing, for each iteration, the number of feasible and infeasible priority orders produced across all 30 trials. The x-axis corresponds to the iteration index, and the y-axis represents the count of trials generating valid or invalid solutions at that iteration. This plot illustrates how quickly each method discovers feasible regions of the search space.
2. **Mean best-so-far convergence curve.** A curve showing, at each iteration, the mean best-so-far objective value computed over the 30 trials using only valid evaluations, together with a one-standard-deviation band ($\mu \pm \sigma$). Invalid samples are excluded because they do not yield meaningful objective values, and including them would distort statistical summaries. This plot therefore reflects the average rate of improvement and the consistency of the best-so-far trajectories across repeated runs, rather than the variability of all raw objective values.

These plots reveal:

- how quickly each method begins to produce valid solutions during the search,
- how efficiently each method improves its best-so-far objective values over iterations.

Numerical Summary Tables. For each agent scale, a table compares all four methods across obstacle types A, B, and C. Each row reports:

- **Effectiveness:** mean values of cost, safety, and the composite objective,
- **Variability:** standard deviation of these metrics across the 30 trials,
- **Computational efficiency:** mean runtime together with its standard deviation.

The numerical tables summarize performance using 30 independent trials for each method. For the random baseline, each trial contributes the best objective value obtained within the same iteration budget used by the optimization methods. For SA, GA, and BO, each trial contributes the final best-so-far objective value at the last iteration. Invalid samples are excluded because they do not correspond to evaluable objective values; including them would distort the statistical summaries. Mean and standard deviation are then computed across the 30 valid trial outcomes.

Consistency of Presentation

Across all scales (10, 20, 30, 40 agents), figures appear in the same fixed format:

1. Environment Layouts (3 side-by-side layouts: types A, B, C)
2. Random Sampling Visualization (3×2 grid)
3. Optimization Visualization (3×2 grid)
4. Numerical Table (full width)

4.3 Medium-Scale Environments

This section evaluates two medium-scale scenarios consisting of 10 agents and 20 agents. For each scenario, start and goal positions are fixed across all experiments, while obstacle configurations vary among types A, B, and C.

4.3.1 10-Agent Environment

(1) Environment Layout. The 10-agent scenario is defined on a fixed grid of size 7×7. Figure 4.1 shows the agent placement and obstacle layouts for types A, B, and C.

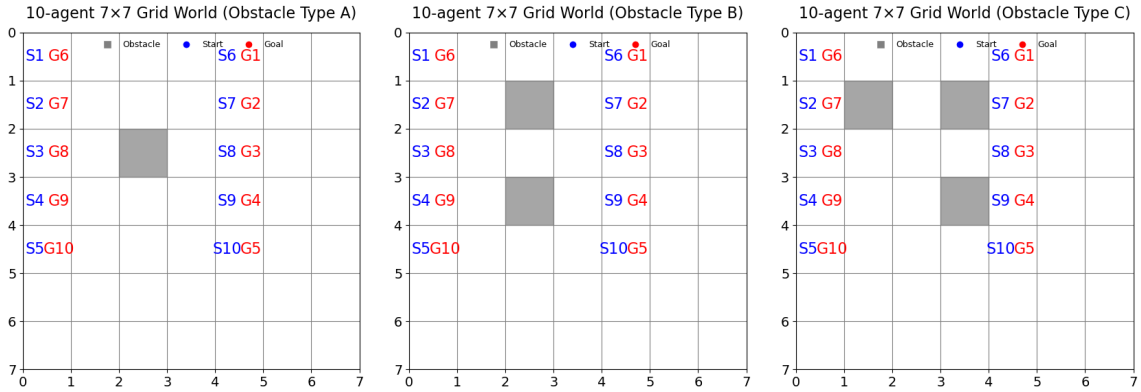


Figure 4.1: Environment layouts for the 10-agent scenario (obstacle types A, B, C).

(2) Random Sampling Results. The random sampling feasibility structure and distribution of valid objective values are shown in Figure 4.2.

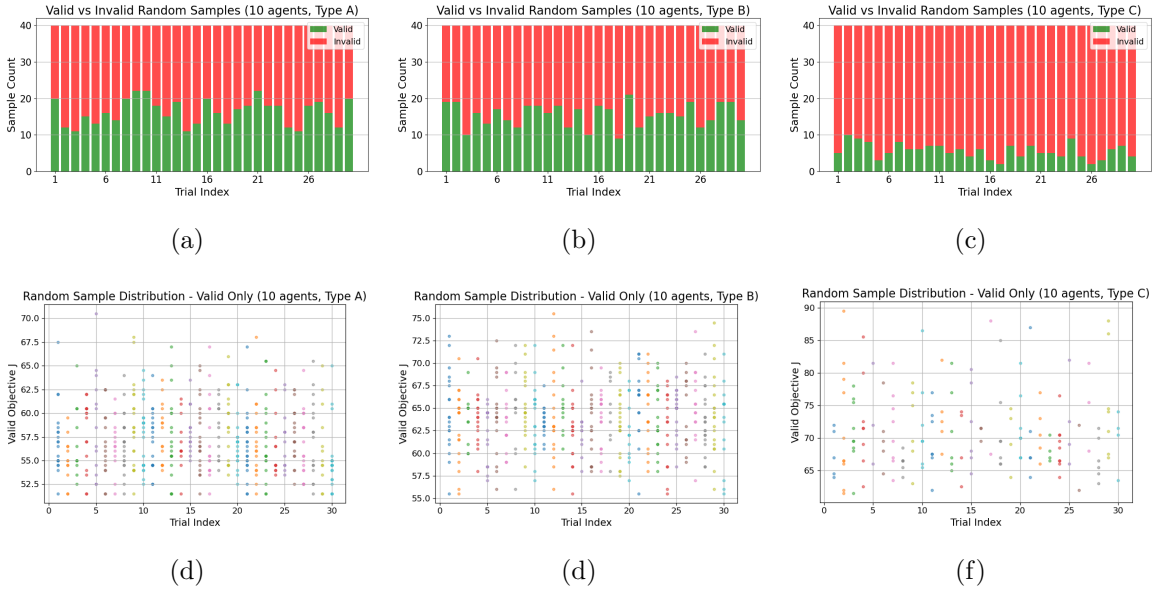
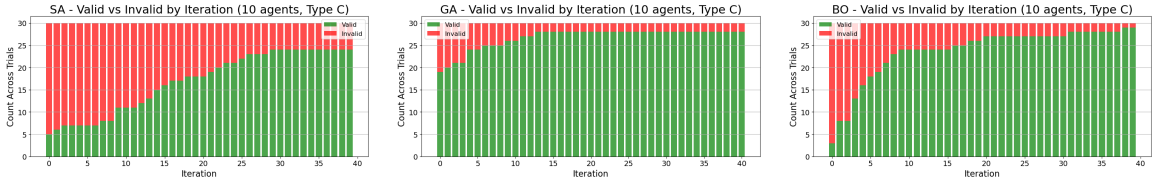
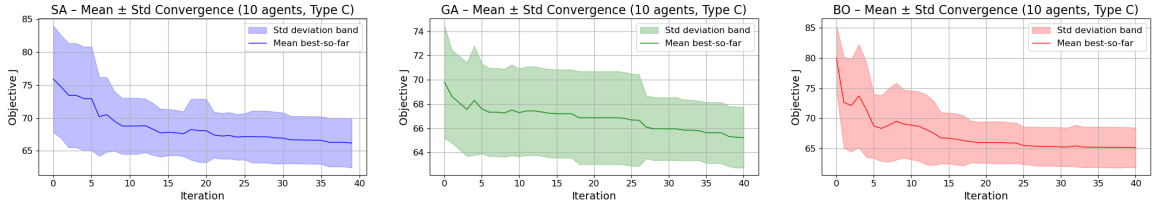


Figure 4.2: Random sampling results for the 10-agent scenario: (a–c) valid–invalid histograms; (d–f) valid-only objective distributions.

(3) Convergence Behavior (Type C). Figure 4.3 summarizes search feasibility and convergence trends for SA, GA, and BO under obstacle type C.



(a) SA validity (b) GA validity (c) BO validity



(d) SA convergence (e) GA convergence (f) BO convergence

Figure 4.3: Optimization results for the 10-agent scenario under obstacle type C: (a–c) feasibility structure; (d–f) mean best-so-far convergence curves with one-standard-deviation bands.

(4) Numerical Results Across Obstacle Types. Table 4.1 reports the performance of all methods across obstacle types A, B, C.

Obstacles	Method	Cost	Safety	Objective	Time (s)
Type A (Low)	Random	63.733 ± 1.209	40.967 ± 1.354	52.350 ± 0.877	0.000 ± 0.000
	SA	65.067 ± 2.707	40.833 ± 1.714	52.950 ± 1.872	0.018 ± 0.002
	GA	64.700 ± 1.735	40.833 ± 1.485	52.767 ± 1.070	0.104 ± 0.010
	BO	63.700 ± 1.159	40.733 ± 1.236	52.217 ± 0.919	2.809 ± 0.075
Type B (Medium)	Random	72.700 ± 3.268	43.733 ± 2.792	58.217 ± 1.754	0.001 ± 0.000
	SA	73.267 ± 3.881	44.367 ± 3.016	58.817 ± 3.185	0.025 ± 0.003
	GA	72.700 ± 3.132	44.067 ± 3.502	58.383 ± 2.619	0.145 ± 0.013
	BO	71.067 ± 2.097	43.000 ± 2.251	57.033 ± 1.821	2.810 ± 0.088
Type C (High)	Random	85.233 ± 5.512	45.267 ± 6.049	65.250 ± 1.905	0.001 ± 0.000
	SA	88.125 ± 6.386	44.250 ± 5.158	66.188 ± 3.749	0.202 ± 0.159
	GA	85.821 ± 4.368	44.643 ± 5.802	65.232 ± 2.502	0.554 ± 0.341
	BO	84.793 ± 5.511	45.448 ± 5.834	65.121 ± 3.311	3.364 ± 0.341

Table 4.1: Performance comparison cross obstacle types for the 10-agent scenario.

4.3.2 20-Agent Environment

(1) Environment Layout. The 20-agent scenario is defined on a fixed grid of size 15×15. Figure 4.4 shows the agent placement and obstacle layouts for types A, B, and C.

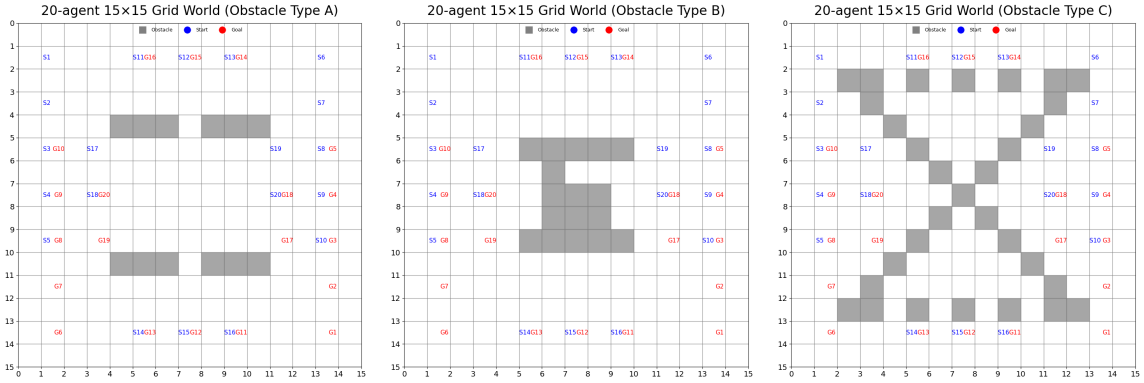
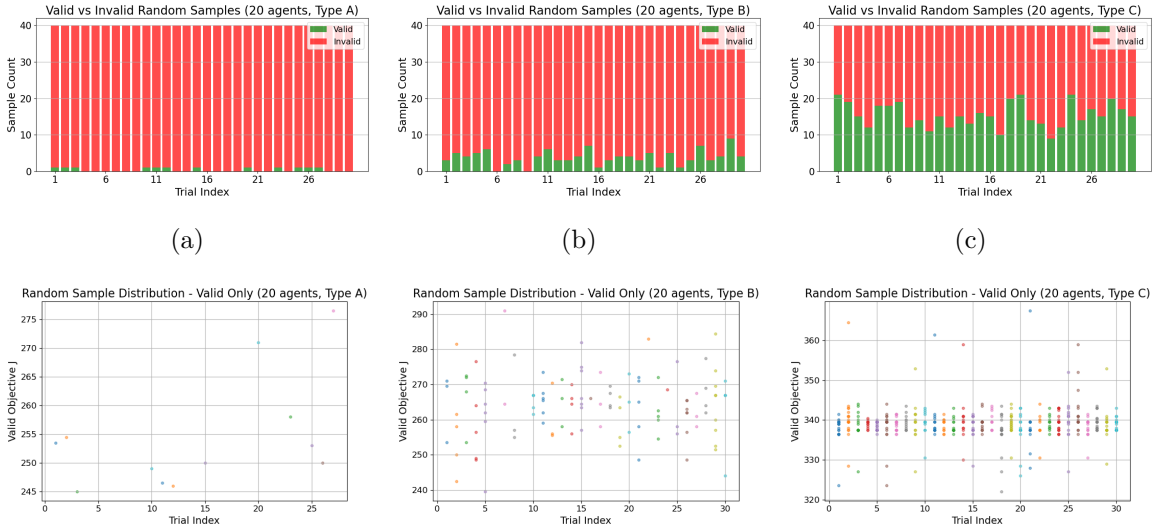


Figure 4.4: Environment layouts for the 20-agent scenario (obstacle types A, B, C).

(2) Random Sampling Results. The random sampling feasibility structure and distribution of valid objective values are shown in Figure 4.5.

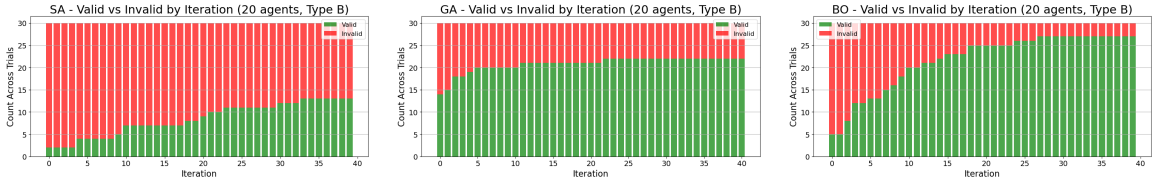


(a) (b) (c)

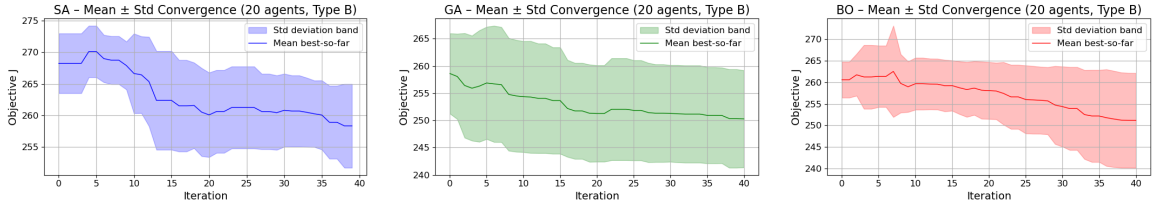
(d) (d) (f)

Figure 4.5: Random sampling results for the 20-agent scenario: (a–c) valid–invalid histograms; (d–f) valid-only objective distributions.

(3) Convergence Behavior (Type B). Figure 4.6 summarizes search feasibility and convergence trends for SA, GA, and BO under obstacle type B.



(a) SA validity (b) GA validity (c) BO validity



(d) SA convergence (e) GA convergence (f) BO convergence

Figure 4.6: Optimization results for the 20-agent scenario under obstacle type B: (a–c) feasibility structure; (d–f) mean best-so-far convergence curves with one-standard-deviation bands.

(4) Numerical Results Across Obstacle Types. Table 4.2 reports the performance of all methods across obstacle types A, B, C.

Obstacles	Method	Cost	Safety	Objective	Time (s)
Type A (Low)	Random	360.000 ± 3.821	154.400 ± 14.016	257.200 ± 7.679	0.007 ± 0.001
	SA	360.667 ± 0.943	145.667 ± 5.558	253.167 ± 2.321	0.263 ± 0.009
	GA	360.100 ± 2.508	144.200 ± 11.025	252.150 ± 5.996	1.577 ± 0.066
	BO	358.600 ± 2.154	152.600 ± 21.463	255.600 ± 11.182	4.398 ± 0.036
Type B (Medium)	Random	394.655 ± 6.076	110.862 ± 21.383	252.759 ± 9.359	0.010 ± 0.001
	SA	392.692 ± 6.244	123.923 ± 17.618	258.308 ± 6.661	0.367 ± 0.033
	GA	395.409 ± 6.058	105.136 ± 18.447	250.273 ± 8.922	2.444 ± 0.278
	BO	394.704 ± 6.364	107.593 ± 25.029	251.148 ± 10.997	4.648 ± 0.284
Type C (High)	Random	530.567 ± 10.442	133.267 ± 19.035	331.917 ± 5.265	0.038 ± 0.004
	SA	523.667 ± 4.974	149.630 ± 12.187	336.648 ± 4.324	1.420 ± 0.065
	GA	526.567 ± 9.746	142.400 ± 18.888	334.483 ± 5.139	8.939 ± 0.828
	BO	530.400 ± 12.422	133.767 ± 27.029	332.083 ± 8.290	5.553 ± 0.110

Table 4.2: Performance comparison cross obstacle types for the 20-agent scenario.

4.4 Large-Scale Environments

This section evaluates two large-scale scenarios consisting of 30 agents and 40 agents. For each scenario, start and goal positions are fixed across all experiments, while obstacle configurations vary among types A, B, and C.

4.4.1 30-Agent Environment

(1) Environment Layout. The 30-agent scenario is defined on a fixed grid of size 20×20. Figure 4.7 shows the agent placement and obstacle layouts for types A, B, and C.

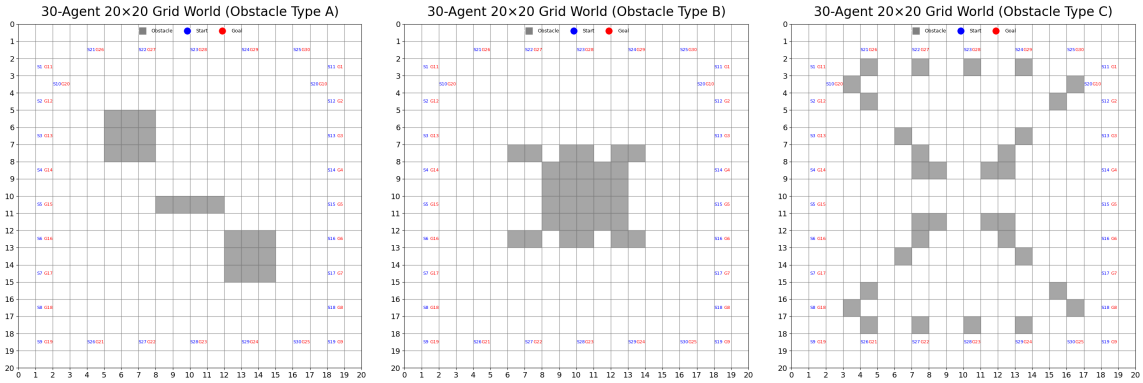
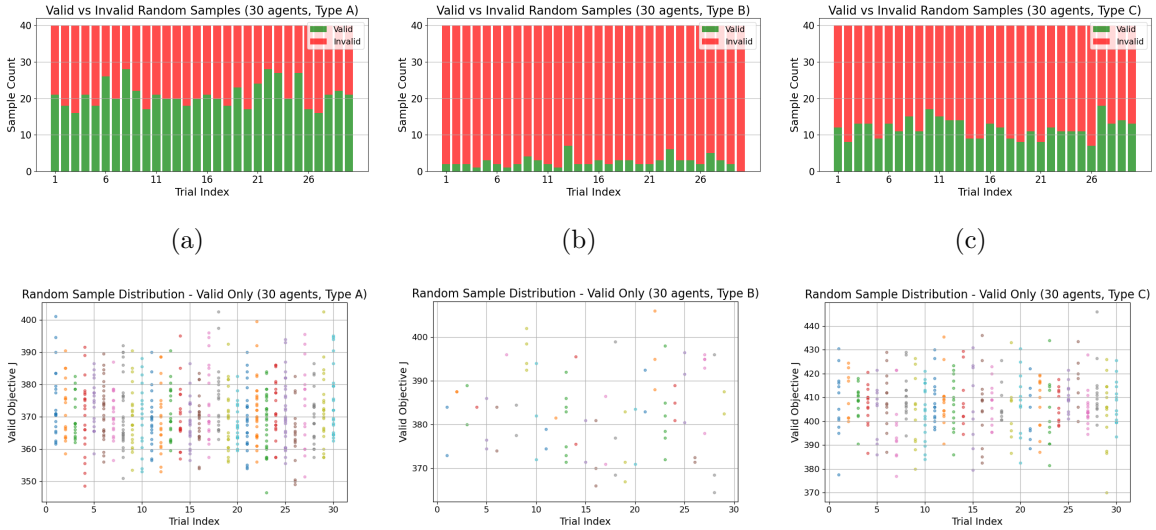


Figure 4.7: Environment layouts for the 30-agent scenario (obstacle types A, B, C).

(2) Random Sampling Results. The random sampling feasibility structure and distribution of valid objective values are shown in Figure 4.8.

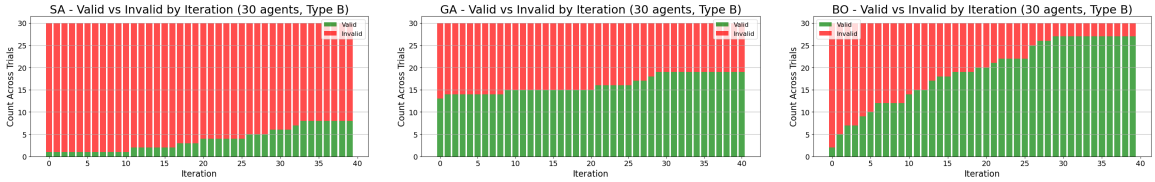


(a) (b) (c)

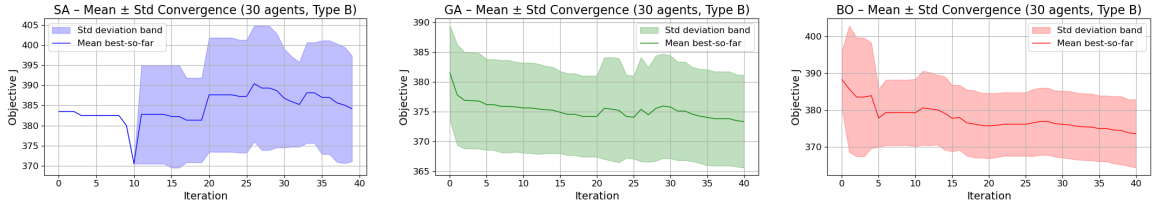
(d) (d) (f)

Figure 4.8: Random sampling results for the 30-agent scenario: (a–c) valid–invalid histograms; (d–f) valid-only objective distributions.

(3) Convergence Behavior (type B). Figure 4.9 summarizes search feasibility and convergence trends for SA, GA, and BO under obstacle type B.



(a) SA validity (b) GA validity (c) BO validity



(d) SA convergence (e) GA convergence (f) BO convergence

Figure 4.9: Optimization results for the 30-agent scenario under obstacle type B: (a–c) feasibility structure; (d–f) mean best-so-far convergence curves with one-standard-deviation bands.

(4) Numerical Results Across Obstacle Types. Table 4.3 reports the performance of all methods across obstacle types A, B, C.

Obstacles	Method	Cost	Safety	Objective	Time (s)
Type A (Low)	Random	608.400 ± 2.185	100.833 ± 6.817	354.617 ± 3.537	0.010 ± 0.001
	SA	609.808 ± 2.829	114.923 ± 16.592	362.365 ± 8.750	0.402 ± 0.022
	GA	608.500 ± 2.964	100.667 ± 9.924	354.583 ± 5.461	2.491 ± 0.137
	BO	608.933 ± 3.812	98.900 ± 6.493	353.917 ± 3.314	4.652 ± 0.142
Type B (Medium)	Random	628.897 ± 2.998	121.345 ± 12.965	375.121 ± 7.194	0.014 ± 0.001
	SA	631.625 ± 3.903	136.750 ± 23.215	384.188 ± 13.143	0.531 ± 0.025
	GA	628.737 ± 3.126	118.000 ± 14.567	373.368 ± 7.759	3.287 ± 0.096
	BO	628.111 ± 3.201	118.963 ± 17.811	373.537 ± 9.270	4.991 ± 0.195
Type C (High)	Random	649.167 ± 3.184	127.500 ± 13.251	388.333 ± 6.901	0.014 ± 0.001
	SA	647.650 ± 3.182	133.550 ± 22.635	390.600 ± 11.138	0.515 ± 0.024
	GA	648.667 ± 3.978	126.400 ± 16.409	387.533 ± 8.427	3.330 ± 0.177
	BO	648.100 ± 3.807	131.833 ± 17.340	389.967 ± 8.676	4.843 ± 0.176

Table 4.3: Performance comparison cross obstacle types for the 30-agent scenario.

4.4.2 40-Agent Environment

(1) Environment Layout. The 40-agent scenario is defined on a fixed grid of size 25×25. Figure 4.10 shows the agent placement and obstacle layouts for types A, B, and C.

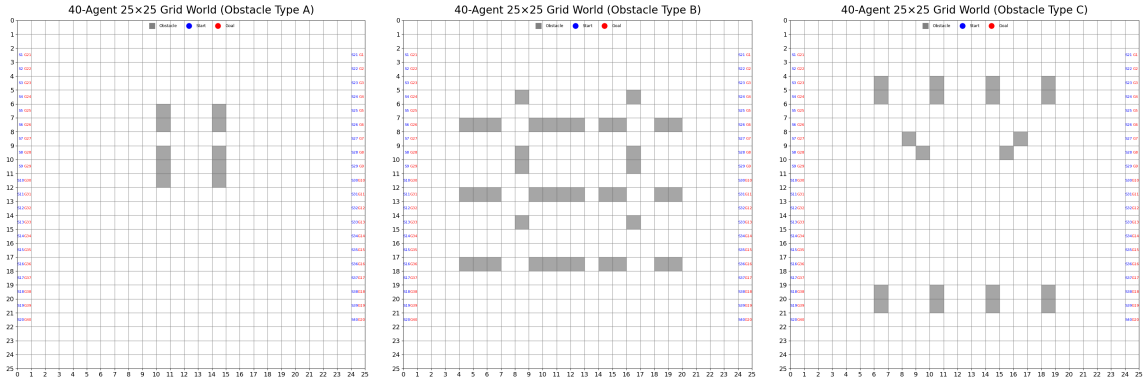
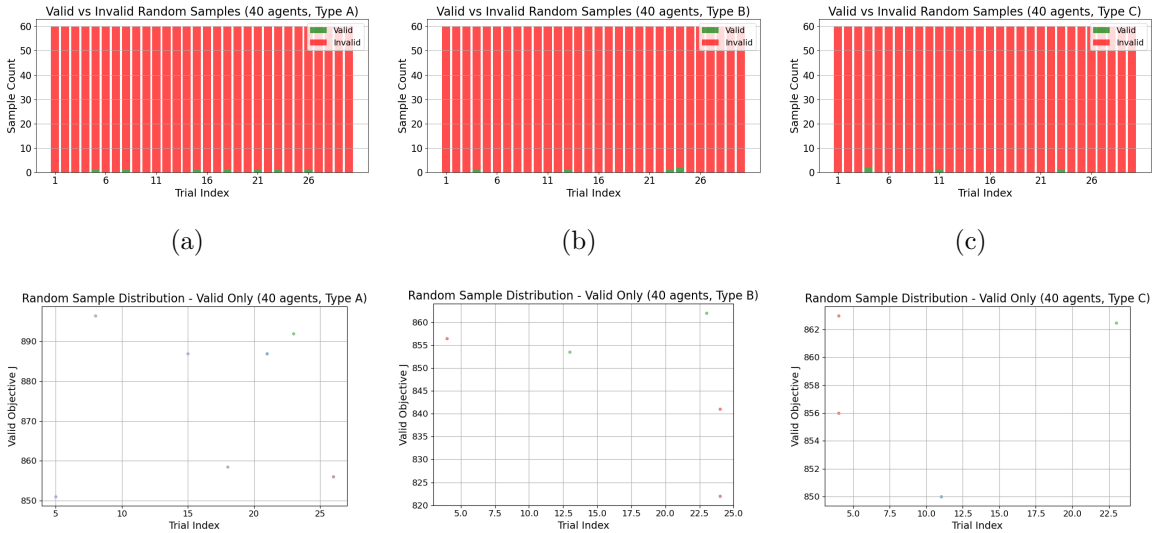


Figure 4.10: Environment layouts for the 40-agent scenario (obstacle types A, B, C).

(2) Random Sampling Results. The random sampling feasibility structure and distribution of valid objective values are shown in Figure 4.11.

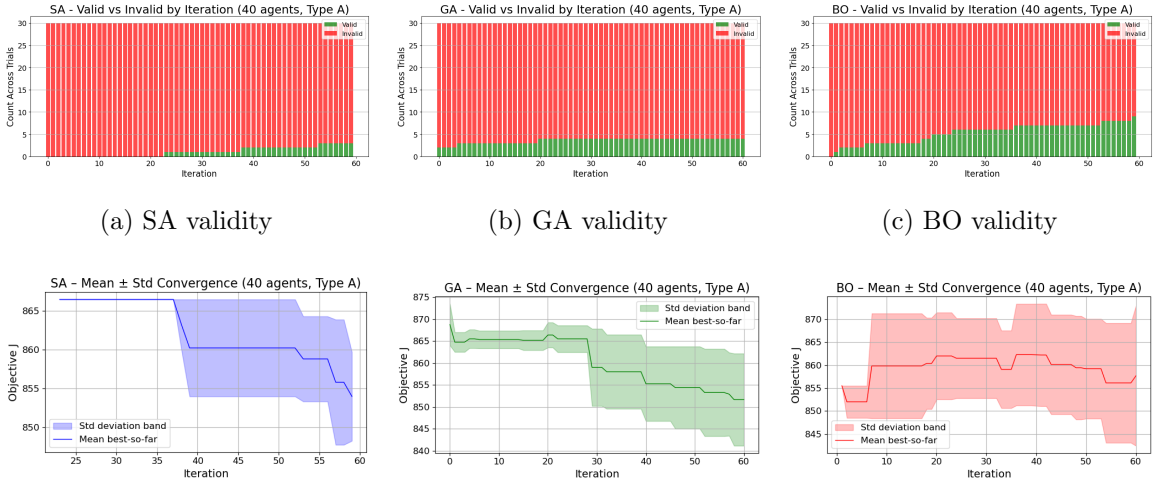


(a) (b) (c)

(d) (d) (f)

Figure 4.11: Random sampling results for the 40-agent scenario: (a–c) valid–invalid histograms; (d–f) valid-only objective distributions.

(3) Convergence Behavior (type A). Figure 4.12 summarizes search feasibility and convergence trends for SA, GA, and BO under obstacle type A.



(a) SA validity (b) GA validity (c) BO validity

(d) SA convergence (e) GA convergence (f) BO convergence

Figure 4.12: Optimization results for the 40-agent scenario under obstacle type A: (a–c) feasibility structure; (d–f) mean best-so-far convergence curves with one-standard-deviation bands.

(4) Numerical Results Across Obstacle Types. Table 4.4 reports the performance of all methods across obstacle types A, B, C.

Obstacles	Method	Cost	Safety	Objective	Time (s)
Type A (Low)	Random	1079.500 ± 5.795	666.167 ± 23.205	872.833 ± 14.226	0.018 ± 0.002
	SA	1080.667 ± 4.110	627.333 ± 15.521	854.000 ± 5.715	0.926 ± 0.022
	GA	1075.750 ± 4.710	627.500 ± 23.712	851.625 ± 10.526	6.382 ± 0.545
	BO	1080.333 ± 7.165	635.000 ± 30.717	857.667 ± 15.252	20.653 ± 1.146
Type B (Medium)	Random	1084.375 ± 7.158	621.250 ± 37.302	852.812 ± 20.486	0.016 ± 0.001
	SA	1075.000 ± 0.000	634.000 ± 0.000	854.500 ± 0.000	0.861 ± 0.000
	GA	1079.000 ± 1.414	577.000 ± 32.342	828.000 ± 15.636	4.710 ± 0.316
	BO	1079.667 ± 4.190	590.667 ± 16.540	835.167 ± 10.119	21.332 ± 1.419
Type C (High)	Random	1076.333 ± 3.300	690.000 ± 31.791	883.167 ± 17.041	0.015 ± 0.000
	SA	1073.000 ± 0.000	577.000 ± 0.000	825.000 ± 0.000	0.889 ± 0.000
	GA	1071.667 ± 1.700	643.667 ± 21.638	857.667 ± 11.441	4.998 ± 0.292
	BO	1072.333 ± 3.543	655.333 ± 26.437	863.833 ± 13.493	19.970 ± 0.459

Table 4.4: Performance comparison cross obstacle types for the 40-agent scenario.

Chapter 5

Discussion and Conclusion

5.1 Interpretation of Results

Across all evaluated environments, the random–priority distributions show that the priority–permutation space contains both highly feasible and highly infeasible regions, together with large variability in achievable objective values. This confirms that priority ordering is not a trivial choice: it strongly influences path feasibility, congestion, and overall execution quality. Therefore, selecting priority orders should be treated as a structured optimization problem rather than a heuristic or fixed decision.

The optimization methods evaluated in this study—SA, GA, and BO—are not expected to produce globally optimal permutations. Instead, their role is to *systematically steer the search away from poor regions of the permutation space* while consistently identifying feasible and higher-quality priority structures.

5.1.1 Simulated Annealing

Across all environments, SA exhibits a gradual and sometimes irregular improvement pattern. The convergence curves do not show a sharp early decrease; instead, the best-so-far objective typically fluctuates during the initial iterations as the method explores the search space with a high temperature. Improvement tends to accumulate more slowly, with noticeable progress only after several iterations once the acceptance probability begins to decrease. Even in later stages, SA maintains visible variability, reflecting its single-trajectory stochastic nature.

Overall, SA consistently avoids the worst regions of the permutation space but improves

at a moderate pace, with final solutions showing greater trial-to-trial variability than GA or BO.

5.1.2 Genetic Algorithm

The Genetic Algorithm exhibits more stable behavior across trials due to its population-based structure. By maintaining and recombining multiple candidate priority orders, GA preserves partial structures that appear beneficial while still introducing diversity through mutation. This allows the search to progress gradually without relying on a single trajectory. However, as with any fixed-operator GA, performance depends on the chosen crossover and mutation strategies, and the algorithm may plateau when the population becomes homogeneous. In general, GA provides stable improvement but not guaranteed superiority in solution quality.

5.1.3 Bayesian Optimization

Bayesian Optimization behaves differently from SA and GA because it is guided by a surrogate model that estimates the objective landscape. The convergence curves show that early iterations often contain noticeable fluctuations as BO explores the space and updates its model. Once sufficient observations are collected, the best-so-far values gradually stabilize, although the improvement trend is not always strictly monotonic.

Across all environments, BO consistently avoids the worst regions of the permutation space and provides steady improvement, but generally does not outperform GA. Its shaded variance bands indicate moderate trial-to-trial variability—less than SA but higher than GA. BO also incurs noticeably higher computational cost due to repeated surrogate updates.

Overall, BO offers a model-driven and reliable form of exploration, but its effectiveness is limited by the complexity of the permutation space and the modest iteration budget used in this study.

5.2 Limitations

While the results demonstrate that SA, GA, and BO all provide improvements over random priority selection, several limitations should be noted:

- **Run-to-run variability.** All three algorithms show noticeable variability across trials, reflecting the stochastic nature of permutation search. The study therefore cannot

make strong claims about consistent superiority of any method.

- **Fixed parameter settings.** Parameter values for SA, GA, and BO were held constant across all environments. No tuning was performed for specific grid sizes or obstacle structures. The reported results therefore reflect baseline behavior rather than case-optimized configurations.
- **Limited sample size per environment.** Although multiple trials were conducted, the dataset is not large enough to support detailed statistical comparisons across algorithms or obstacle patterns. The study focuses on qualitative tendencies rather than quantitative rankings.
- **Local operators.** The permutation operators used (swaps, 3-cycle rotations, and insert moves) are simple and not optimized for large or highly constrained permutations. Different operators could change the relative behavior of the algorithms.
- **Feasibility limits of the planner.** Some environments become near-infeasible for the underlying time-expanded planner, independent of the optimization method. In these cases, all algorithms may return invalid results, reflecting environmental difficulty rather than optimization failure.

5.3 Future Work

Future research may explore the following directions:

Richer neighborhood operators. Operators such as long-segment rotations, block exchanges, or learned mutation models may enhance exploration and reduce stagnation.

Advanced SA and GA variants. Adaptive temperature schedules, island-model GAs, and multi-objective extensions could provide more reliable convergence patterns.

Joint optimization with dynamic replanning. Co-optimizing priority ordering with dynamic replanning may improve feasibility and reduce congestion in dense environments.

5.4 Conclusion

This study examines optimization-based priority selection for prioritized multi-agent path planning using SA, GA, and BO under fixed parameter settings. The experiments demonstrate that randomly selected priority permutations can lead to outcomes ranging from highly inefficient or infeasible solutions to very efficient and balanced ones. This highlights

the importance of treating priority selection as an explicit optimization problem rather than a fixed or heuristic choice.

Across all environments, SA, GA, and BO each provide improvement over random sampling by systematically avoiding poor regions of the permutation space and identifying priority structures that yield feasible and higher-quality solutions.

SA provides early-stage correction, though the extent of improvement and the final performance vary across stochastic runs. GA offers stable and population-driven progress, generally improving reliably across trials. BO demonstrates structured, model-guided exploration with robust avoidance of catastrophic solutions, albeit at higher computational cost.

Overall, the results support the viability of applying general-purpose optimization methods to priority-order selection in prioritized MAPP. The methods provide predictable advantages over unguided sampling, while the study also highlights opportunities for further refinement in operator design, model tuning, and integration with dynamic replanning frameworks.

References

[1] R. Stern, N. Sturtevant, A. Felner, S. Koenig *et al.*, "Multi-agent pathfinding: Definitions, variants, and benchmarks," in *Proceedings of the Twelfth International Symposium on Combinatorial Search (SoCS)*, 2019.

[2] M. Erdmann and T. Lozano-Pérez, "On multiple moving objects," *Algorithmica*, vol. 2, pp. 477–521, 1987.

[3] D. Silver, "Cooperative pathfinding," in *Proceedings of the First Artificial Intelligence and Interactive Digital Entertainment Conference (AIIDE)*, 2005.

[4] M. Čáp, P. Novák, A. Kleiner, and M. Selecký, "Prioritized planning algorithms for trajectory coordination of multiple mobile robots," *IEEE Transactions on Automation Science and Engineering*, vol. 12, no. 3, pp. 835–849, 2015.

[5] S. Zhang, J. Li, T. Huang, S. Koenig, and B. Dilkina, "Learning a priority ordering for prioritized planning in multi-agent path finding," in *Proceedings of the Fifteenth International Symposium on Combinatorial Search (SoCS)*, 2022, pp. 208–216.

[6] H. Ma, D. Harabor, P. Stuckey, J. Li, and S. Koenig, "Searching with consistent prioritization for multi-agent path finding," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2019.

[7] M. Bennewitz, W. Burgard, and S. Thrun, "Optimizing schedules for prioritized path planning of multi-robot systems," in *Proceedings of the IEEE International Conference on Robotics and Automation.* IEEE, 2001, pp. 271–276.

[8] ——, "Finding and optimizing solvable priority schemes for decoupled path planning techniques for teams of mobile robots," *Robotics and Autonomous Systems*, vol. 41, pp. 89–99, 2002.

[9] S.-H. Chan, R. Stern, A. Felner, and S. Koenig, "Greedy priority-based search for suboptimal multi-agent path finding," in *Proceedings of the Sixteenth International Symposium on Combinatorial Search (SoCS)*, 2023, pp. 11–19.

[10] P. Tian, J. Ma, and D.-M. Zhang, "Application of the simulated annealing algorithm to the combinatorial optimisation problem with permutation property: An investigation of generation mechanism," *European Journal of Operational Research*, vol. 118, pp. 81–94, 1999.

[11] H.-F. Wang and K.-Y. Wu, "Hybrid genetic algorithm for optimization problems with permutation property," *Computers & Operations Research*, vol. 31, pp. 2453–2471, 2004.

[12] A. Deshwal, S. Belakaria, J. R. Doppa, and D. H. Kim, "Bayesian optimization over permutation spaces," 2021.

[13] M. de Sales Guerra Tsuzuki, T. de Castro Martins, and F. K. Takase, "Robot path planning using simulated annealing," in *Proceedings of the 12th IFAC/IFIP/IFORS/IEEE/IMS Symposium Information Control Problems in Manufacturing (INCOM)*, 2006, pp. 175–180.

[14] W. Shi, Z. He, W. Tang, W. Liu, and Z. Ma, "Path planning of multi-robot systems with boolean specifications based on simulated annealing," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 6091–6098, 2022.

[15] K. Shi, L. Yang, Z. Wu, B. Jiang, and Q. Gao, "Multi-robot dynamic path planning with priority based on simulated annealing," *Journal of the Franklin Institute*, vol. 362, no. 1, 2025.

[16] S. Alarabi, T. Lei, M. Santora, C. Luo, and T. Sellers, "Multi-robot path planning using potential field-based simulated annealing approach," in *Proceedings of the SPIE Unmanned Systems Technology XXVI*, 2024.

[17] H. Qu, K. Xing, and T. Alexander, "An improved genetic algorithm with co-evolutionary strategy for global path planning of multiple mobile robots," *Neurocomputing*, vol. 120, pp. 509–517, 2013.

[18] M. Nazarahari, E. Khanmirza, and S. Doostie, "Multi-objective multi-robot path planning in continuous environment using an enhanced genetic algorithm," *Expert Systems with Applications*, vol. 115, pp. 106–120, 2019.

[19] R. Kala, "Multi-robot path planning using co-evolutionary genetic programming," in *Expert Systems with Applications*, vol. 39, 2012, pp. 3817–3831.

[20] G. Ryou, E. Tal, and S. Karaman, "Cooperative multi-agent trajectory generation with modular bayesian optimization," in *Proceedings of the Robotics: Science and Systems (RSS)*, MIT. New York City, NY, USA: Robotics: Science and Systems Foundation, June 2022.

[21] Z. Wang, M. Zhang, S. Liang, S. Yu, C. Zhang, and S. Du, "A multi-objective path-planning approach for multi-scenario urban mobility needs," *Algorithms*, vol. 18, no. 1, 2025.

[22] F. Ho and S. Nakadai, "Preference-based multi-objective multi-agent path finding," in *Proceedings of the 22nd International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*. London, United Kingdom: International Foundation for Autonomous Agents and Multiagent Systems (IFAAMAS), 2023, pp. 2000–2002.

Chapter A

Glossary and Acronyms

Care has been taken in this thesis to minimize the use of jargon and acronyms, but this cannot always be achieved. This appendix defines jargon terms in a glossary, and contains a table of acronyms and their meaning.

A.1 Glossary

- **Multi-Agent Path Planning (MAPP)** – The problem of computing collision-free paths for multiple agents that move synchronously in a shared environment, where each agent must travel from a start to a goal position without colliding in space-time.
- **Prioritized (Priority) Planning** – A decoupled MAPP strategy in which agents plan sequentially according to a priority order; higher-priority agents commit their paths first, and their trajectories become constraints for lower-priority agents.
- **Dynamic Goal Blocking** – In sequential MAPP, an agent that reaches its goal remains there and acts as a temporary obstacle for all remaining agents.
- **Reservation Table** – A time–space data structure that records which grid cells are occupied at each timestep, preventing vertex, edge, and timing conflicts.
- **Workload / Cost** – The total path length (or energy expenditure) incurred by an agent during its planned trajectory.
- **Safety / Congestion Cost** – A quantity measuring delays and interference experienced by agents, usually through waiting time and congestion penalties.
- **Fairness (Workload Variance)** – A measure of imbalance in workloads across agents; lower variance corresponds to more balanced task distribution.

- **Makespan** – The total completion time of a multi-agent plan, defined as the time when the last agent reaches its goal.
- **Simulated Annealing (SA)** – A stochastic local-search optimization algorithm that probabilistically accepts worse solutions to escape local minima.
- **Genetic Algorithm (GA)** – A population-based optimization algorithm that evolves candidate solutions through selection, crossover, and mutation.
- **Bayesian Optimization (BO)** – A model-based optimization method that uses a surrogate model (e.g., Gaussian Process) and acquisition function to select informative samples.
- **Permutation Space** – The set of all possible priority orders for N agents; the space has size $N!$.
- **Kendall Tau Distance** – A metric that measures the difference between two permutations based on the number of pairwise ordering disagreements.

A.2 Acronyms

Table A.1: Acronyms

Acronym	**Meaning**
MAPP	Multi-Agent Path Planning
COF	Composite Objective Function
SA	Simulated Annealing
GA	Genetic Algorithm
BO	Bayesian Optimization
A*	A-star Path Planning Algorithm
GP	Gaussian Process